SnAI IA-1 — COMPLETE KIT

Exam card

- **Paper:** SnAI IA-1 — Computer Fundamentals, OS, Git & Version Control, Computer Networks (Modules 1–10).
- **Syllabus source:** Slack `academic-announcement-26`, 19 Oct, 6:20 PM (verbatim in Part 0).
- **Portal:** course `tjj12ybxo9tt`, subject `eq7lnkj6lo7s` (SnAI-B).
- **Tonight's order:** Part 1 concept sheet (60 min) → Module 10 networks re-read (15 min) → Practice set A timed (30 min) → MCQs (15 min) → sleep.

PART 0 — SYLLABUS (verbatim)

Modules: 1 Computer Systems (intro, types, IPOS, applications) · 2 Hardware & Software (components, software types: OS/application/utility) · 3 CLI (terminal, file & directory mgmt, package mgmt, flags/options/operators) · 4 Interfaces & Architecture (CLI/GUI/API, system architecture) · 5 Operating Systems (intro, process, memory, device, filesystem mgmt, UI & shell, OS architecture, file ownership & permissions, file paths, os module) · 6 Computer Software (compilers, interpreters, difference) · 7 Number Representation & Data Encoding (decimal/binary/hex, conversion, encoding methods) · 8 Git & Version Control (intro, Git, essential commands, tracking changes, workflow) · 9 GitHub (intro, repo mgmt, collaboration, shared repos) · 10 Computer Networks (intro, LAN/MAN/WAN, DNS, packet switching, protocols, routers, client-server, network tools).

> "Questions may also be asked from related concepts and topics that extend beyond what was explicitly covered in class."

PART 1 — CONCEPT COURSE (what IA-1 actually asks)

M1. Computer systems

- **Computer** = electronic device that takes **input** → **processes** → **output** → **stores** (**IPOS**). Every computer follows IPOS.
- **Types:** supercomputer, mainframe, server, desktop/laptop, mobile, embedded, workstation. Difference is **scale, purpose, cost** — not principle.
- **Data centre vs phone:** same capabilities (compute/store/network) at different **scale and primary purpose**.
- **Hardware components:** CPU (ALU + control unit + registers), RAM (volatile, temporary working memory), ROM/SSD/HDD (non-volatile storage), motherboard, PSU, input/output devices.
- **Von Neumann idea:** instructions and data live in the same memory; CPU fetches → decodes → executes.

M2. Hardware & software

- **Software** = set of instructions. Types: **system software** (OS, drivers, utilities), **application software** (browser, editor), **utility software** (antivirus, backup, disk cleaner).
- Firmware sits between hardware and OS (BIOS/UEFI).

M3. CLI

- Terminal = text interface to the shell (bash/zsh). Commands are programs; **flags** modify them (`-l`, `--human-readable`); `|`, `>`, `>>`, `&&`, `;` are **operators**.
- Core commands: `pwd`, `ls -la`, `cd`, `mkdir -p`, `touch`, `cp -r`, `mv`, `rm -r` / `rm -rf` (careful), `cat`, `less`, `head`, `tail`, `wc -l`, `find`, `grep -r`, `sed -i`, `awk`, `chmod`, `chown`, `du -sh`, `df -h`, `tar -czf` / `tar -xzf`.
- **Package management:** Debian/Ubuntu `apt update`, `apt install`, `apt remove`; RHEL `dnf` / `yum`; macOS `brew`; Python `pip`.
- **Redirection:** `>` overwrite, `>>` append, `2>` errors, `2>&1` both, `|` pipe stdout to next command.

M4. Interfaces & architecture

- **CLI** (fast, scriptable), **GUI** (mouse, discoverable), **API** (program-to-program contract).
- Architecture layers: hardware → kernel → shell/OS services → applications → user.

M5. Operating systems

- **OS** = resource manager + abstraction layer. Functions: process, memory, device, filesystem management, security, UI/shell.
- **Process states:** new → ready → running → waiting → terminated; **context switch**; **scheduling** (FCFS, SJF, Round Robin, priority).
- **Memory:** RAM vs virtual memory, paging, swapping, fragmentation (internal/external).
- **Device management:** drivers, spooling, buffering, interrupt handling.
- **Filesystem:** inode, directory tree, mount points, journaling; ext4/NTFS/APFS.
- **File permissions (octal):** r=4, w=2, x=1 → `755` = rwxr-xr-x, `644` = rw-r--r--, `600` = rw-------, `640` = rw-r-----; `chmod u+x`, `chmod 750 dir`; **directories need x to enter**.
- **Ownership:** user/group/others; `chown user:group file`.
- **Paths:** absolute (`/home/user/x`) vs relative (`./x`, `../x`); `PATH` = colon-separated dirs searched for commands (`which`, `echo $PATH`).
- **os module (Python):** `os.getcwd()`, `os.listdir()`, `os.mkdir()/makedirs()`, `os.rename()`, `os.remove()`, `os.rmdir()`, `os.path.exists/join/basename`, `os.environ`, `os.system()`.
- **Shell types:** sh, bash, zsh; shell = command interpreter.

M6. Compiler vs interpreter

	Compiler	Interpreter
Translation	whole program at once	line by line
Output	machine-code/object file (exe)	none — executes directly
Speed	faster execution	slower execution

	Compiler	Interpreter
Errors	reported after full scan	stops at first error
Examples	C, C++, Go (`gcc hello.c -o hello`)	Python, Ruby, JS

- **Shebang** `#!/usr/bin/env python3` + `chmod +x` makes a script directly runnable.

M7. Number systems & encoding

- **Decimal (base 10), binary (2), octal (8), hex (16: 0-9,A-F).**
- Conversions: divide-by-base for decimal→binary/hex; multiply-by-2 for fraction; group 4 bits = 1 hex digit.

- 156 → `10011100` → `9C`; 255 → `11111111` → `FF`; binary `1011` = 11 = `B`.

- **Encoding:** ASCII (7-bit, 128 chars), extended ASCII (8-bit, 256), **Unicode/UTF-8** (all scripts, variable 1-4 bytes).
- Text file = bytes + encoding; wrong encoding = mojibake.

M8. Git

- Version control = history of changes; **Git** = distributed VCS (every clone has full history).
- **Three areas:** working directory → staging area (`git add`) → repository (`git commit`).
- Commands: `git init`, `git status`, `git add <file>` / `git add .`, `git commit -m "msg"`, `git log --oneline`, `git diff`, `git restore` / `git checkout -- file`, `git branch`, `git switch -c feat`, `git merge`, `git remote add origin <url>`, `git push -u origin main`, `git pull`, `git clone <url>`.
- Commit = snapshot + message + author + timestamp + parent hash. HEAD = current commit pointer.
- **Undo:** `git restore <file>` (discard working change), `git reset --soft/--mixed/--hard`, `git revert <hash>`.

M9. GitHub

- Remote hosting for Git; **fork** (your copy), **branch**, **pull request** (propose merge), **review**, **merge**, **issues**.
- Collaboration flow: `git pull` → branch → commit → push → open PR → review → merge → pull again.
- `git remote -v`, `git fetch`, `git remote set-url`.

M10. Computer networks

- **Network** = devices sharing resources. **LAN** (room/building), **MAN** (city), **WAN** (country/world; internet = largest WAN).
- **Topologies:** star, bus, ring, mesh. **Media:** copper, fibre, wireless.
- **IP/MAC:** IP = logical address, MAC = hardware address; IPv4 32-bit, IPv6 128-bit.
- **DNS** = phonebook: domain → IP (A, AAAA, CNAME, MX records). Resolution: browser cache → resolver → root → TLD → authoritative.
- **Packet switching:** data split into packets, routed independently, reassembled; vs circuit switching.
- **Protocols:** TCP (reliable, ordered, handshake), UDP (fast, no guarantee), HTTP/HTTPS (web, 80/443), FTP (21), SSH (22), SMTP (25), DNS (53), DHCP (auto IP).
- **OSI 7 layers:** Physical, Data-link, Network, Transport, Session, Presentation, Application. **TCP/IP 4:** Link, Internet, Transport, Application.
- **Router** = forwards packets between networks (uses IP, routing table); **switch** = same LAN (MAC); **hub** = dumb broadcast.

- **Client-server** vs peer-to-peer. **Network tools:** `ping` (reachability/RTT), `traceroute` (path), `nslookup` / `dig` (DNS), `netstat` / `ss` (connections/ports), `curl` / `wget` (HTTP), `ip a` / `ifconfig` (interfaces), `ssh`, `scp`.

M11. Exam-day one-liners

- IPOS · r=4 w=2 x=1 · 755 dir / 644 file / 600 secret · `chmod +x` + shebang to run a script
- Compiler = whole file, interpreter = line by line
- 4 bits = 1 hex digit; ASCII 7-bit; UTF-8 = Unicode encoding
- git: work → `add` → `commit` → `push`; `pull` before you push
- DNS = name→IP; TCP reliable, UDP fast; router between networks, switch inside one
- LAN room, MAN city, WAN world

PART 2 — ALL 53 PORTAL QUESTIONS, WITH ANSWERS

Module 3 — Command-Line Interface (CLI)

Terminal, file/dir management, package management, flags & operators. — 4 questions.

Q1. Advanced File Management

40/40

You are helping your team clean up and organize project files.

Steps to follow:

Create a folder called project-files.

Inside it, create three files:

report.docx

summary.docx

draft.docx

Make a copy of report.docx and name it report_backup.docx.

Create a new folder called old. Move draft.docx into old/.

Rename summary.docx to final_summary.docx.

Copy final_summary.docx into the old/ folder as well.

Delete report_backup.docx.

Final structure should look like this:

```
project-files/
├── report.docx
├── final_summary.docx
└── old/
    ├── draft.docx
```

```
└── final_summary.docx
```

Answer

Create the folder and files, then copy/move/rename/delete exactly as listed.

```
mkdir project-files && cd project-files   # create and enter working folder

touch report.docx summary.docx draft.docx # create the three files

cp report.docx report_backup.docx         # backup copy

mkdir old                                 # folder for archived files

mv draft.docx old/                        # move draft into old/

mv summary.docx final_summary.docx        # rename summary

cp final_summary.docx old/                # duplicate final summary into old/

rm report_backup.docx                     # remove the backup copy
```

Check: `ls -R` should show `project-files/report.docx`, `final_summary.docx`, and `old/{draft.docx,final_summary.docx}` — no `report_backup.docx`.

Q2. Create a Project Folder Structure

40/40

You have recently joined a backend team that develops services for an e-commerce platform.

To ensure consistency across projects, the team uses an internal command called create-service to generate the standard structure for new API services.

During the onboarding of several new developers, the team noticed that newly generated services are missing important files and some directories are named incorrectly.

The engineering manager asks you to investigate and repair the scaffold generator before the next batch of services is created.

Tasks:

Locate the create-service command.

Inspect the script used by the command.

Fix all issues in the script.

Ensure the command can be executed.

Run the command to generate a new service.

Expected Service Structure:

service/

├── controllers/

├── models/

├── routes/

├── middleware/

├── server.js

└── README.md

Expected README Content:

API Service

Expected server.js Content:

```
console.log("Server started");




Requirements:

Directory names must exactly match the expected structure.

The middleware directory must be present.

README.md must exist inside the service directory.

server.js must exist inside the service directory.

The contents of README.md must exactly match the expected text.

The contents of server.js must exactly match the expected text.

The command should execute successfully after your fixes.
```

Answer

Locate the command, rewrite its broken script so it always builds the exact tree, make it executable, then run it.

```
type -a create-service                # locate the command in PATH

script=$(command -v create-service)

cat "$script"                         # inspect existing/broken script

# Repair: replace body with a correct generator

sudo tee "$script" >/dev/null <<'EOF'

#!/usr/bin/env bash

set -e

mkdir -p service/controllers service/models service/routes service/middleware

printf 'API Service\n' > service/README.md

printf 'console.log("Server started");\n' > service/server.js

EOF

sudo chmod +x "$script"               # ensure it is executable

create-service                        # generate service/ in current directory
```

Check: `ls -R service` shows all five dirs/files; `cat service/README.md service/server.js` matches the two required lines exactly.

Q3. Recover the Emergency Access Token

40/40

You are assisting the Site Reliability Engineering (SRE) team during a production incident.

An emergency access token is required to restore a critical service. The token was intentionally stored somewhere inside a deeply nested backup directory structure for security reasons.

The backup system has created many nested folders containing many files. Most of these files are not readable due to permission restrictions.

The SRE team remembers only one thing:

The emergency token is stored in the only readable text file within the backup directory tree.

Your task is to locate the readable file, inspect its contents, and recover the token.

Tasks:

Explore the directory structure.

Find the readable file inside the backup directory tree.

Create a file named token.txt in the current directory containing only the token value.

Example Token Format:

ACCESS_TOKEN_VALUE=abc123xyz789

Expected Output File:

abc123xyz789

Requirements:

Do not modify any existing files.

token.txt must contain only the token value.

No additional text or whitespace should be present.

Answer

Find the only readable file containing `ACCESS_TOKEN=`, strip the prefix, and write just the value.

```
dir=$(find . -maxdepth 3 -type d -iname '*backup*' | head -n1); echo "$dir"

find "$dir" -type f -readable -exec grep -l 'ACCESS_TOKEN=' {} +   # locate token file

f=$(find "$dir" -type f -readable -exec grep -l 'ACCESS_TOKEN=' {} + | head -n1)

sed -n 's/^ACCESS_TOKEN=//p' "$f" | head -n1 | tr -d '\n' > token.txt
```

Check: `cat token.txt` prints only `abc123xyz789` (no `ACCESS_TOKEN=`, no trailing newline).

Q4. Locate and Extract the Recovery Key

```
40/40


A production server experienced a critical failure overnight. Before shutting down, the backup
system created multiple backup folders and scattered diagnostic files across them.


Your team needs to restore access, but the recovery key was saved in only one backup file. The file
name and location are unknown.


Search through the backup directory tree, find the recovery key, and save only the key value
(without RECOVERY_KEY=) into a file named key.txt in the current directory.


Task:


Locate the recovery key hidden somewhere inside the backups directory.

Extract only the recovery key value.

Save the key value into a file named key.txt in the current directory.


For example, if a file contains:


RECOVERY_KEY=7fK92mXqL8pR3vNz


then key.txt should contain:


7fK92mXqL8pR3vNz


The recovery key may be located in any file within the backups directory tree.
```

Answer

Search the `backups` tree for `RECOVERY_KEY=`, then write only the value to `key.txt`.

```
find backups -type f -exec grep -l 'RECOVERY_KEY=' {} + 2>/dev/null   # locate key file

f=$(find backups -type f -exec grep -l 'RECOVERY_KEY=' {} + 2>/dev/null | head -n1)

sed -n 's/^RECOVERY_KEY=//p' "$f" | head -n1 | tr -d '\n' > key.txt
```

Check: `cat key.txt` prints only `7fK92mXqL8pR3vNz` (prefix removed, no trailing newline).

Contest 1 (CLI missions)

Multi-step shell missions — same commands, longer chains. — 5 questions.

Q5. Detective Shubham: The Hidden Network

0/40

You are playing the role of Detective Shubham, a cyber investigator.

You have received information about a secret digital operation.

The entire operation follows a strict folder structure, and your job is to recreate it exactly using Linux CLI commands only.

You are currently inside the box/ directory.

Your Mission Steps :-

Step 1

Create a main folder named: OperationX

This folder will act as the control center.

Step 2

Move inside the OperationX folder.

Step 3

Inside OperationX, create three folders:

intel

agents

logs

Step 4

Inside the intel folder, create two folders:

documents

images

Step 5

Inside the agents folder, create three folders:

alpha

bravo

```
charlie


Step 6

Inside the logs folder, create two folders:

system

security




Expected Final Folder Structure

OperationX

├── intel

│   ├── documents

│   └── images

├── agents

│   ├── alpha

│   ├── bravo

│   └── charlie

└── logs

    ├── system

    └── security

na

na

na

na
```

Answer

Create `OperationX` inside `box/` and build the whole tree in one `mkdir -p`.

```
mkdir OperationX && cd OperationX

mkdir -p intel/documents intel/images \

         agents/alpha agents/bravo agents/charlie \

         logs/system logs/security
```

Check: `find . -type d | sort` lists all 9 folders and nothing else; `pwd` ends in `box/OperationX`.

Q6. New Recruit

40/40

You are playing the role of a new recruit at a Company.

Before you can access any real case files, you must set up your personal workspace exactly the way the agency requires — using Linux CLI commands only.

You are currently inside the box/ directory.

Your Mission Steps:

Step 1

Create a main folder named: MyWorkspace

Step 2

Move inside the MyWorkspace folder.

Step 3

Inside MyWorkspace, create two folders:

notes

tasks

Step 4

Inside the notes folder, create an empty file named: day1.txt

Expected Final Folder Structure

```
MyWorkspace
├── notes
│   └── day1.txt
└── tasks
```

Answer

Create the workspace, its two folders, and an empty `day1.txt`.

```
mkdir MyWorkspace && cd MyWorkspace

mkdir notes tasks

touch notes/day1.txt
```

Check: `find . | sort` shows `MyWorkspace/notes/day1.txt` and `MyWorkspace/tasks`; `ls -l notes/day1.txt` shows size 0.

Q7. A Digital Investigator

0/80

You are playing the role of Detective Senorita, a digital investigator.

You've just received an encrypted folder named CaseFiles from an anonymous source.

Your task is to organize the evidence, record your findings, and prepare a quick summary report — using Linux commands only.

Your Mission Steps

Create a new folder named CaseFiles.

Move inside the CaseFiles folder.

Inside this folder, create three empty files:

suspects.txt

evidence.txt

summary.txt

Add the following lines to suspects.txt, one line at a time without overwriting previous content:

John – Seen near crime scene

Maya – Works at nearby cafe

Raj – Security guard on duty

Add the following lines to evidence.txt, one line at a time without overwriting previous content:

CCTV footage found

Fingerprints on glass

Combine suspects.txt and evidence.txt into summary.txt. The suspects must appear first, followed by the evidence.

Display the contents of summary.txt.

Expected Final Report

John – Seen near crime scene

Maya – Works at nearby cafe

Raj – Security guard on duty

CCTV footage found

Fingerprints on glass

Answer

Create `CaseFiles`, write each finding with `>>` (append), then concatenate suspects first and evidence second.

```
mkdir CaseFiles && cd CaseFiles

touch suspects.txt evidence.txt summary.txt

printf '%s\n' 'John – Seen near crime scene'   >> suspects.txt

printf '%s\n' 'Maya – Works at nearby cafe'   >> suspects.txt

printf '%s\n' 'Raj – Security guard on duty'  >> suspects.txt

printf '%s\n' 'CCTV footage found'            >> evidence.txt

printf '%s\n' 'Fingerprints on glass'         >> evidence.txt

cat suspects.txt evidence.txt > summary.txt

cat summary.txt
```

Check: `cat summary.txt` prints the three suspect lines followed by the two evidence lines, in that order.

Q8. CID

0/120

You are playing the role of a CID wrapping up a compromised operation.

The old operation folder OldBase contains leftover, disorganized files from a previous mission. Your job is to inspect, reorganize, rename, back up, and clean up the entire structure using Linux CLI commands only — exactly as instructed. This mission tests every command in your toolkit.

You are currently inside the box/ directory, and OldBase/ already exists with the following starting structure:

```
OldBase
├── draft.txt        (contains the text: TOP SECRET)
├── temp
│   └── junk.txt
└── old_logs
    └── log1.txt
```

Your Mission Steps:

Step 1

Create a new main folder named: NewBase

Step 2

Move inside the NewBase folder.

Step 3

Inside NewBase, create three folders:

reports

archive

backup

Step 4

Move back OUT of NewBase (using cd ..) so you're inside box/ again and print working directory.

Step 5

Copy the file OldBase/draft.txt into NewBase/reports/ (keep the same filename).

Step 6

Move (rename) NewBase/reports/draft.txt to NewBase/reports/final_report.txt.

Step 7

Add a new line of text — MISSION COMPLETE — to the end of NewBase/reports/final_report.txt without deleting its existing content.

Step 8

Display the contents of NewBase/reports/final_report.txt in the terminal to confirm it now contains BOTH lines.

Step 9

Move the entire OldBase/old_logs folder into NewBase/backup/.

Step 10

Delete the file OldBase/temp/junk.txt (the file only, not the folder).

Step 11

Delete the now-empty OldBase/temp folder using the command meant for removing empty directories.

Step 12

Delete the entire OldBase folder and everything remaining inside it in a single command.

Expected Final Folder Structure

```
NewBase
├── reports
│   └── final_report.txt   (contains: TOP SECRET \n MISSION COMPLETE)
├── archive
└── backup
```

```
    └── old_logs
        └── log1.txt


(OldBase should no longer exist at all.)
```

Answer

Build `NewBase`, copy/rename/append the report, relocate `old_logs`, then remove `OldBase` piece by piece.

```
mkdir NewBase && cd NewBase

mkdir reports archive backup

cd ..                                  # back to box/

pwd

cp OldBase/draft.txt NewBase/reports/

mv NewBase/reports/draft.txt NewBase/reports/final_report.txt

printf 'MISSION COMPLETE\n' >> NewBase/reports/final_report.txt

cat NewBase/reports/final_report.txt   # must show TOP SECRET + MISSION COMPLETE

mv OldBase/old_logs NewBase/backup/

rm OldBase/temp/junk.txt               # delete file only

rmdir OldBase/temp                     # only works once empty

rm -rf OldBase                         # remove whole old tree
```

Check: `ls -R NewBase` matches the expected tree, `cat NewBase/reports/final_report.txt` has both lines, and `test ! -e OldBase && echo gone` prints `gone`.

Q9. Mr. Robot

0/120

You are playing the role of a Mr. Robot closing out a classified operation known as Mirage.

Nothing about this operation is straightforward — every file has a decoy, every path has a twist, and one wrong move will corrupt the evidence. Follow the mission exactly.

You are currently inside the box/ directory, and MirageBase/ already exists with the following starting structure:

```
MirageBase
├── vault
│   └── secret.txt        (contains: CODE ALPHA)
├── decoys
│   ├── decoy1.txt
│   └── decoy2.txt
└── archive
    └── old.txt            (contains: ARCHIVE DATA)
```

Your Mission Steps:

Step 1

Create a new folder named: Mirage

Step 2

Move inside the Mirage folder.

Step 3

Inside Mirage, create three folders:

vault_copy

confirmed

discarded

Step 4

Move inside the vault_copy folder.

Step 5

From here, copy MirageBase/vault/secret.txt into your current folder (vault_copy), keeping the same filename. ⚠️ Remember — you are now two folders deep, so a plain ../ won't reach box/.

Step 6

Add a new line — VERIFIED — to the end of the secret.txt now sitting in vault_copy. ⚠️ The original content (CODE ALPHA) must still be there afterward. One wrong symbol here will silently destroy it.

Step 7

Display the contents of this secret.txt in the terminal to confirm both lines are present.

Step 8

Move back out of vault_copy, and then out of Mirage, so that you are back inside box/ — using cd .. twice, not any shortcut.

Step 9

Confirm your current location using the appropriate command.

Step 10

Move (not copy) MirageBase/archive/old.txt into Mirage/confirmed/.

Step 11

Delete the two files inside MirageBase/decoys — decoy1.txt and decoy2.txt — one at a time.

Step 12

Now remove the MirageBase/decoys folder using the command meant strictly for empty directories. ⚠️ If you try this before Step 11 is fully done, it will refuse to work — that's the point.

Step 13

Finally, delete the entire MirageBase folder — which still contains the original vault/secret.txt you never removed, plus an now-empty archive folder — in a single command.

```
Expected Final Folder Structure


Mirage
├── vault_copy
│   └── secret.txt      (contains: CODE ALPHA \n VERIFIED)
├── confirmed
│   └── old.txt          (contains: ARCHIVE DATA)
└── discarded             (empty)


(MirageBase should no longer exist anywhere.)
```

Answer

Build `Mirage`, copy and verify the secret from two levels deep, move the archive file, then dismantle `MirageBase` safely.

```
mkdir Mirage && cd Mirage

mkdir vault_copy confirmed discarded

cd vault_copy

cp ../../MirageBase/vault/secret.txt .   # ../../ reaches box/ from Mirage/vault_copy

printf 'VERIFIED\n' >> secret.txt        # append, keep CODE ALPHA

cat secret.txt                           # both lines must appear

cd ..                                    # out of vault_copy

cd ..                                    # out of Mirage, back in box/

pwd                                      # confirm box/

mv MirageBase/archive/old.txt Mirage/confirmed/

rm MirageBase/decoys/decoy1.txt

rm MirageBase/decoys/decoy2.txt

rmdir MirageBase/decoys                  # succeeds only when empty

rm -rf MirageBase                        # single command removes the rest
```

Check: `ls -R Mirage` matches expected; `cat Mirage/vault_copy/secret.txt` shows `CODE ALPHA` + `VERIFIED`; `test ! -e MirageBase && echo gone` prints `gone`.

Module 5/6 — OS, Filesystem, Permissions, PATH, os module

Permissions, ownership, paths, os module. — 5 questions.

Q10. Audit Trail Setup

```
10/20



You are given the following folder structure inside the current directory.



audit_trail/

└── access.log



Perform the following operations:



Change the permissions of the audit_trail/ directory so that the owner has full access (read,
write, execute), the group has read and execute only, and others have no access at all.

Inside audit_trail/, create a new file named summary.txt containing the exact text Audit Summary
Generated.

Set the permissions of summary.txt so that the owner can read and write, the group can only read,
and others have no access.
```

Answer

Set the directory to `750` (rwxr-x---), add the summary, then set it to `640` (rw-r-----).

```
chmod 750 audit_trail                        # owner rwx, group r-x, others none

printf 'Audit Summary Generated\n' > audit_trail/summary.txt

chmod 640 audit_trail/summary.txt            # owner rw, group r, others none
```

Check: `ls -ld audit_trail` shows `drwxr-x---`; `ls -l audit_trail/summary.txt` shows `-rw-r-----`; `cat audit_trail/summary.txt` prints `Audit Summary Generated`.

Q11. Three Levels of Access

```
10/20


After a security review, three files in the clearance directory were found with wide-open
permissions. Each file must be tightened to a different clearance level using numeric chmod.


Given Directory Structure:


clearance/
├── public.txt
├── team.txt
└── private.txt


Current Permissions:


All three files: full permissions for everyone.


Tasks to Complete:


public.txt: owner can read and write; group and others can only read.
team.txt: owner can read and write; group can only read; others get nothing.
private.txt: owner can read and write; group and others get nothing.
Do not modify the content of any file.
```

Answer

Apply the three numeric modes: `644`, `640`, `600`.

```
chmod 644 clearance/public.txt    # owner rw, group r, others r
chmod 640 clearance/team.txt      # owner rw, group r, others none
chmod 600 clearance/private.txt   # owner rw, group/others none
```

Check: `ls -l clearance` shows `-rw-r--r--`, `-rw-r-----`, `-rw-------` respectively; contents unchanged.

Q12. Deployment Project Permissions

20/40

You are preparing a shared deployment project where different roles (developers, CI system, general viewers) need different levels of access to different parts of the codebase. You need to set the correct permissions on each part of the project so it is accessed only as intended.

Given Directory Structure:

```
deployment/
├── app/
│   ├── server.py
│   ├── handler.py
│   └── settings.py
├── datasets/
│   ├── train.csv
│   └── test.csv
├── docs/
│   └── OVERVIEW.md
└── bin/
    ├── start.sh
    ├── stop.sh
    └── healthcheck.sh
```

All files currently have default permissions (644 for files, 755 for directories). Apply the following rules:

Python source files (every .py file inside deployment/app/):

Owner: read, write, execute

Group: no access

Others: no access

The datasets/ directory, and every file inside it:

Owner: read, write, execute

Group: read, write, execute

Others: no access

```
docs/OVERVIEW.md:

Owner: read, write

Group: read only

Others: read only

Every shell script (every .sh file inside deployment/bin/):

Owner: read, write, execute

Group: read, execute (no write)

Others: read, execute (no write)


Deliverable: Write the chmod command(s) you would run, one rule at a time, to reach the end state
described above. You may use wildcards (e.g. *.py) instead of listing files individually.


Note: Run your commands from outside the deployment/ directory (i.e. paths should start with
deployment/..., not be run from inside it).
```

Answer

Run four `chmod` rules from the parent directory, using wildcards for each group of files.

```
chmod 700 deployment/app/*.py                              # rwx------

chmod 770 deployment/datasets deployment/datasets/*        # rwxrwx---

chmod 644 deployment/docs/OVERVIEW.md                      # rw-r--r--

chmod 755 deployment/bin/*.sh                              # rwxr-xr-x
```

Check: `ls -ld deployment/datasets` shows `drwxrwx---`; `find deployment -exec stat -c '%a %n' {} +` shows py= `700`, dataset entries= `770`, OVERVIEW= `644`, sh= `755`.

Q13. Purge the Locked Workspace

```
0/40


A corrupted draft is stuck inside a locked workspace. The draft itself is read-only, and the
workspace directory has no write permission. Deleting a file requires write permission on the
directory, not on the file. Delete the corrupted draft, create a clean replacement, and lock the
workspace again.


Given Directory Structure:


workspace/     (Write access removed)

├── old_draft.txt   (Read-only, corrupted)

└── notes.txt


Current Permissions:


workspace/ directory: Write permission removed (chmod 555).

old_draft.txt: Read-only for everyone (chmod 444).


Tasks to Complete:


Delete workspace/old_draft.txt.

Create a new file workspace/final_draft.txt containing exactly Clean copy.

Using chmod, set the permissions of final_draft.txt to read and write for user, read for group and
no permissions for others.

notes.txt must remain untouched.

Restore workspace/ permissions to read and execute for user, group and others.
```

Answer

Temporarily add write on the directory (needed to delete/create entries), replace the draft, then relock.

```
chmod u+w workspace                        # allow changes inside workspace/

rm -f workspace/old_draft.txt

printf 'Clean copy\n' > workspace/final_draft.txt

chmod 640 workspace/final_draft.txt        # owner rw, group r, others none

chmod 555 workspace                        # r-xr-xr-x for all
```

Check: `ls -ld workspace` shows `dr-xr-xr-x`; `ls -l workspace` shows only `final_draft.txt` (`-rw-r-----`) and untouched `notes.txt`; `cat workspace/final_draft.txt` prints `Clean copy`.

Q14. Data Breach Response

0/60

You have been tasked with handling Data Breach Response, a secure directory of evidence and reports from an active security incident. Some files and directories currently have restricted permissions that you must work around without permanently weakening security.

Given Directory Structure:

```
breach_response/
├── evidence/
│   └── log_dump.txt
├── secure/
├── reports/
│   └── incident_summary.txt
└── classified.txt
```

Pre-existing Permissions:

evidence/ directory — write permission removed (chmod 555)

log_dump.txt — read-only (chmod 444)

incident_summary.txt — read-only (chmod 444)

classified.txt — no permissions (chmod 000)

Actions to Perform:

Copy a file into breach_response/:

Copy evidence/log_dump.txt to breach_response/log_dump.txt.

Move the restricted directory:

Move the entire evidence/ directory into secure/, resulting in secure/evidence/.

evidence/ currently has no write permission (555). Since moving a directory into a new parent requires updating its own metadata, you'll need to temporarily add write permission to evidence/ before moving it.

After the move, restore secure/evidence/ to permission 555.

```
Edit the read-only report:

Append the exact line Breach confirmed to reports/incident_summary.txt.

incident_summary.txt currently has no write permission (444), so you'll need to temporarily adjust
permissions to edit it.

Restore the file permissions to 444 afterward.

Create a new report:

Create secure/forensics_report.txt containing the exact text Forensics Complete.

Set its permissions to 600.

Append to the report again:

Append the exact line Escalated to security team to reports/incident_summary.txt.

As before, temporarily adjust permissions to write, then restore incident_summary.txt to 444
afterward.



Important: Do not modify classified.txt. It must remain unchanged with permission 000.
```

Answer

Copy the log, temporarily add write where needed, move `evidence/`, and restore `444`/`555`/`600` afterwards.

```
cp breach_response/evidence/log_dump.txt breach_response/log_dump.txt



chmod u+w breach_response/evidence

mv breach_response/evidence breach_response/secure/

chmod 555 breach_response/secure/evidence



chmod u+w breach_response/reports/incident_summary.txt

printf 'Breach confirmed\n' >> breach_response/reports/incident_summary.txt

chmod 444 breach_response/reports/incident_summary.txt



printf 'Forensics Complete\n' > breach_response/secure/forensics_report.txt

chmod 600 breach_response/secure/forensics_report.txt



chmod u+w breach_response/reports/incident_summary.txt

printf 'Escalated to security team\n' >> breach_response/reports/incident_summary.txt

chmod 444 breach_response/reports/incident_summary.txt
```

Check: `cat breach_response/reports/incident_summary.txt` shows both appended lines; `ls -ld breach_response/secure/evidence` is `dr-xr-xr-x`; `ls -l breach_response/secure/forensics_report.txt` is `-rw-------`; `stat -c '%a' breach_response/classified.txt` is `000`.

Module 6/7 — Software, Compiler/Interpreter, Number systems

Compiled vs interpreted; decimal/binary/hex + encoding. — 3 questions.

Q15. Execute a Python Script Directly

```
10/20

A teammate has asked you to create a Python script that can be executed directly from the terminal
without typing python3 before the filename.

Create a file named hello.py that prints: Hello from Python

Configure the script so that it can be executed directly from the terminal.

Expected Result:

Running the following command:

./hello.py

should display:

Hello from Python

Hello from Python

The script must be named hello.py.
```

Answer

Add a shebang as the first line, then mark the file executable so the OS uses it as the interpreter path.

```
cat > hello.py <<'EOF'

#!/usr/bin/env python3

print("Hello from Python")

EOF

chmod +x hello.py

./hello.py
```

Check: `./hello.py` prints `Hello from Python`; `ls -l hello.py` shows `-rwxr-xr-x`.

Q16. Resolve the Issue with hello.py

```
10/20

A teammate has handed over a Python script named hello.py and says:

"The script works on my machine, but I can't run it directly from the terminal on yours. Can you
fix it?"



You are not allowed to modify the script itself. Investigate the problem and make the script
executable so it can be run directly from the terminal. Running the following command:

./hello.py

should display:

Hello from Python

Hello from Python

Do not modify the contents of hello.py.

Run the script directly from the terminal using ./hello.py instead of invoking it through the
Python interpreter.
```

Answer

The file lacks the execute bit; only permissions are changed, never the contents.

```
chmod +x hello.py

./hello.py
```

Check: `ls -l hello.py` shows `x` in the permission string and `./hello.py` prints `Hello from Python`.

Q17. Simplify the Compilation Process

```
10/20

Your team frequently compiles a C program before testing it. Instead of typing the compilation
command every time, your manager wants a reusable build script.

Create a shell script named build.sh that compiles hello.c into an executable named hello.

The script should be executable and should automate the build process using terminal commands only.

Expected Result

After creating the script, run both of the following commands:

./build.sh

./hello

The output should be:

Hello from C

Hello from C
```

Answer

Put the `gcc` compile command inside build.sh, make the script executable, then run it and the resulting binary.

```
cat > build.sh <<'EOF'

#!/bin/bash

gcc hello.c -o hello

EOF

chmod +x build.sh

./build.sh

./hello
```

Check: `./hello` prints `Hello from C`; `ls -l hello` shows an executable file (compilation happens once per `./build.sh`).

Module 8 — Git & Version Control

init, add, commit, status, log, checkout, branching, remote, push/pull. — 4 questions.

Q18. Undo Incorrect Changes and Commit the Fix

```
20/20


You accidentally added the wrong line Wrong line to app.txt and have NOT staged it.


Undo the unstaged change using git checkout.


Then add the correct line Correct line to the end of app.txt as a new line, stage the file, and
commit it with the message:


Fix app content
```

Answer

Restore the unstaged file from HEAD, append the correct line, then stage and commit.

```
git checkout -- app.txt

echo "Correct line" >> app.txt

git add app.txt

git commit -m "Fix app content"
```

Check: `git log --oneline -1` shows `Fix app content`; `git show --stat HEAD` lists only app.txt.

Q19. A new team member sets up a repository

```
20/20

You joined a new project. The folder my-app already exists with a file app.py inside it. Navigate
into my-app, initialize Git, stage everything, and make the first commit with message: Project
setup
```

Answer

Enter the existing project folder, initialize the repo, stage everything, and make the initial commit.

```
cd my-app

git init

git add .

git commit -m "Project setup"
```

Check: `git log --oneline` shows one commit `Project setup`; `git status` shows a clean working tree.

Q20. Multiple File Commits and Commit History Review

```
0/20


Repository is already initialized.


The files header.html and footer.html already exist in the repository.


Add some content to header.html, then stage and commit your changes with the message:


Add header


Next, add some content to footer.html, then stage and commit your changes with the message:


Add footer


Finally, run:


git log --oneline


to verify that both commits appear in the commit history.
```

Answer

Edit one file at a time, staging and committing each separately so the history has two distinct commits.

```
echo "<header>Site header</header>" >> header.html

git add header.html

git commit -m "Add header"



echo "<footer>Site footer</footer>" >> footer.html

git add footer.html

git commit -m "Add footer"



git log --oneline
```

Check: `git log --oneline` lists `Add footer` then `Add header`; `git status` is clean.

Q21. Selective Staging with Multiple Commits and Log Review

```
0/40


A Git repository is already initialized, and both README.md and main.py already exist.


Add some content to README.md, stage only README.md, and commit it with the message:


Update readme


Then add some content to main.py, stage only main.py, and commit it with the message:


Update main


Finally, run:


git log --oneline


to view the commit history.
```

Answer

Stage each file by name (not `git add .`) so each commit contains only its intended change.

```
echo "Updated readme content" >> README.md

git add README.md

git commit -m "Update readme"



echo "Updated main content" >> main.py

git add main.py

git commit -m "Update main"



git log --oneline
```

Check: `git log --oneline` shows both `Update main` and `Update readme`; `git show --stat HEAD` lists only main.py.

GitHub & remote workflow

Clone, remote, collaboration. — 4 questions.

Q22. Clone and Verify Before Working

```
0/20

Before starting work on a shared project, your process is: clone https://github.com/org/inventory-
system.git, navigate into inventory-system, verify the remote is set correctly, pull any latest
changes (just to be safe), and check the commit log in compact format.
```

Answer

Clone the given repo, enter it, confirm `origin`, fast-forward with a pull, then view compact history.

```
git clone https://github.com/org/inventory-system.git

cd inventory-system

git remote -v

git pull

git log --oneline
```

Check: `git remote -v` shows `origin` fetch/push URLs; `git status` reports up to date.

Q23. Full Team Collaboration Simulation

```
0/40



You're picking up a teammate's CRM project. Clone https://github.com/teamdev/crm-tool.git and cd
into crm-tool. Someone may have pushed a last-minute commit right after you cloned, so run git pull
to be safe.



The cloned src/contacts.js and src/deals.js are empty stub files — write some real content into
each before staging them (e.g. echo "export const contacts = [];" > src/contacts.js). Do not commit
until you have added content to both files.



Stage only src/contacts.js and src/deals.js — never stage src/internal-notes.js or
build/output.min.js. Commit with message Add CRM core modules. The remote is already set from the
clone, so just push.



Finally, run git log and git remote -v to confirm.
```

Answer

Clone/pull first, add real content to both stub modules, stage only those two paths, then commit and push.

```
git clone https://github.com/teamdev/crm-tool.git

cd crm-tool

git pull

echo "export const contacts = [];" > src/contacts.js

echo "export const deals = [];" > src/deals.js

git add src/contacts.js src/deals.js

git commit -m "Add CRM core modules"

git push

git log --oneline

git remote -v
```

Check: `git status` is clean, `git log -1 --stat` lists only the two files, `git remote -v` shows origin.

Q24. Collaborative Git Workflow Challenge

```
0/40

You are working on a collaborative project in the chat-app directory. Navigate into it. A remote
repository is already configured. Then pull to check if there are any changes on remote or not.
Add content Resolved message issue in client.js, then stage and commit it using the commit message:
"fix: resolve message duplicate issue". Finally push your update to the remote repository.
```

Answer

Enter the existing repo, sync with the remote, edit client.js, then stage, commit, and push.

```
cd chat-app

git pull

echo "Resolved message issue" >> client.js

git add client.js

git commit -m "fix: resolve message duplicate issue"

git push
```

Check: `git log --oneline -1` shows the fix commit; `git status` says up to date with `origin`.

Q25. New Machine, Same Project

```
0/40



You just got a new laptop. Clone https://github.com/yourname/recipe-app.git. Navigate into recipe-
app. Pull (to be safe).



Add some content to app.js, db.js, and helpers/format.js. Do not commit until you have made changes
to all three files. Then stage all three — but NOT helpers/debug.js.



Commit with message Resume recipe app development. Push back.
```

Answer

Clone, pull, modify all three tracked files, stage them by explicit path (excluding debug.js), then commit and push.

```
git clone https://github.com/yourname/recipe-app.git

cd recipe-app

git pull

echo "// app logic" >> app.js

echo "// db logic" >> db.js

echo "// format helper" >> helpers/format.js

git add app.js db.js helpers/format.js

git commit -m "Resume recipe app development"

git push
```

Check: `git show --stat HEAD` lists exactly the three files (no helpers/debug.js); `git status` is clean.

Practice set A (16 questions)

Mixed CLI/OS/Git drills — do these timed. — 16 questions.

Q26. Reorganizing the HR Folder Structure

0/20

Kartik has just joined a new company as a data analyst. However, on his first day, his manager asks him to help the HR team organise their folder structure. Kartik’s task is to reorganise the HR team's existing folder hierarchy according to the instructions provided by the manager.

Kartik must work with various terminal commands to complete this task.

Kartik begins with the following folder structure inside the

home directory:

```
home

    ├── employees/

    │    ├── john_doe/

    │    │    ├── profile.txt

    │    │    └── contract.txt

    │    ├── jane_smith/

    │    │    ├── profile.txt

    │    │    └── contract.txt

    ├── temp_documents/

    │    └── guidelines.txt

    ├── management/
```

```
    │    └── confidential.txt

    ├── to_process/

    └── completed/
```

Kartik's manager gives him the following tasks to reorganise the structure:

The files inside the employees/ directory are not named properly. The manager wants Kartik to rename the contract.txt files to match the employee's name for better organization. Rename john_doe/contract.txt to john_doe/john_contract.txt. Rename jane_smith/contract.txt to jane_smith/jane_contract.txt.

There is no backup folder in the current HR structure. Kartik needs to create a backup/ directory right under the home/ folder and copy the entire management/ folder (along with its files) into this new backup directory.

Kartik needs to create a file in the to_process/ directory that contains some text for the team to process later. Create a file named tasks.txt inside the to_process/ folder and add the following content: Organize all employee records and ensure contracts are securely stored.

The management/confidential.txt file contains sensitive information, and it should only be accessed by the owner (the HR manager). Kartik needs to change the file permissions so that only the owner can read and write to it, and no other users should have access to it.

The temp_documents/ folder is no longer needed. Kartik needs to remove the entire folder and its contents.

Help Kartik shine in his new job by completing these tasks for him.

Answer

Run from inside `home/`: rename the contracts, back up management, add the task note, lock the confidential file, delete temp_documents.

```
mv employees/john_doe/contract.txt employees/john_doe/john_contract.txt

mv employees/jane_smith/contract.txt employees/jane_smith/jane_contract.txt

mkdir -p backup

cp -r management backup/

printf 'Organize all employee records and ensure contracts are securely stored.\n' >
to_process/tasks.txt

chmod 600 management/confidential.txt

rm -rf temp_documents
```

Check: `ls -R` shows `backup/management/confidential.txt`, no `temp_documents/`; `ls -l management` shows `-rw-------`.

Q27. ClientVault Reorganization

0/60

You have been tasked with reorganizing ClientVault, a secure directory holding sensitive client files. The folder structure is already in place, but some files and directories currently have restricted permissions. Your job is to copy, move, and update files while respecting (and correctly restoring) those permissions.

Given Directory Structure:

```
client_vault/
├── originals/
│   └── contract.pdf
├── staging/
├── audit/
│   └── access.log
└── notes.txt
```

Pre-existing Permissions:

originals/ directory — write permission removed (chmod 555)

contract.pdf — read-only (chmod 444)

access.log — write permission removed (chmod 444)

notes.txt — read permission removed (chmod 000)

Actions to Perform:

Copy a file into client_vault/:

Copy contract.pdf from originals/ into client_vault/ (result: client_vault/contract.pdf).

Move a restricted directory:

Move the entire originals/ directory into staging/ (result: staging/originals/, containing contract.pdf).

originals/ currently has no write permission (555). Since moving a directory into a new parent requires updating its own metadata, you'll need to temporarily add write permission to originals/ before moving it.

After the move, staging/originals/ must be restored to 555.

```
Edit a read-only log file:

Append the exact line Client data reviewed to audit/access.log.

access.log currently has no write permission (444), so you'll need to temporarily adjust
permissions to edit it.

After appending, restore access.log to 444.

Create a new report file:

Inside staging/, create a new file named staging_report.txt.

Its content should be exactly: Staging Review Complete.

Set its permissions to 600 (owner: read+write; group: no access; others: no access).

Append to the log file a second time:

Append the exact line Second review complete to audit/access.log.

As before, temporarily adjust permissions to write, then restore access.log to 444 afterward.



Important: notes.txt is not part of any task above. Do not touch it — at the end of the task it
must still be exactly as it started, with permissions 000.
```

Answer

From the parent of `client_vault/`, temporarily add owner write so 555/444 files can be moved/edited, then restore exact modes.

```
cp client_vault/originals/contract.pdf client_vault/contract.pdf



chmod u+w client_vault/originals

mv client_vault/originals client_vault/staging/originals

chmod 555 client_vault/staging/originals



chmod u+w client_vault/audit/access.log

printf 'Client data reviewed\n' >> client_vault/audit/access.log

chmod 444 client_vault/audit/access.log



printf 'Staging Review Complete\n' > client_vault/staging/staging_report.txt

chmod 600 client_vault/staging/staging_report.txt



chmod u+w client_vault/audit/access.log

printf 'Second review complete\n' >> client_vault/audit/access.log

chmod 444 client_vault/audit/access.log
```

Check: `stat -c '%a %n' client_vault/staging/originals client_vault/audit/access.log client_vault/staging/staging_report.txt client_vault/notes.txt` -> 555, 444, 600, 000.

Q28. Directory Manager

```
0/20

You have been assigned the task of setting up the directory structure for a new project. The
project requires the following directory hierarchy:



project/

├── src/

│   ├── components/

│   ├── services/

│   └── utils/

├── tests/

│   ├── unit/

│   └── integration/

└── docs/

    ├── api/

    └── guides/




Your task is to create this entire directory structure using a single mkdir command. Ensure that
all directories are created in one go, without needing to navigate into each parent directory.



Example : To create multiple nested directories, you can utilise this command format

mkdir -p /var/www/html/project/{css,js,images}

This piece of code created the following directory structure :



/var/www/html/project/

                    ├── css

                    ├── js

                    └── images



Write the mkdir command you would use to accomplish this.
```

Answer

One `mkdir -p` using brace expansion for each sibling group.

```
mkdir -p project/src/{components,services,utils} project/tests/{unit,integration}
project/docs/{api,guides}
```

Check: `find project -type d | sort` lists all 8 subdirectories under `project/`.

Q29. Directory cleaning

0/20

You are managing a project directory that has accumulated unnecessary files and directories over time. Your task is to clean up the directory by removing specific files and folders that are no longer needed.

Given the following directory structure:

```
project/
├── src/
│   ├── old_components/
│   ├── temp. txt
│   └── services/
├── tests/
│   ├── old_tests/
│   ├── unit/
│   └── integration/
├── docs/
│   ├── old_api/
│   ├── api/
│   └── guides/
└── README. md
```

You need to remove the following:

The entire old_components directory inside src/.

The temp. txt file inside src/.

The entire old_tests directory inside tests/.

The entire old_api directory inside docs/.

Write the rm command(s) you would use to accomplish this task.

Answer

Remove the three directories recursively and the single temp file.

```
rm -rf src/old_components

rm -f src/temp.txt

rm -rf tests/old_tests

rm -rf docs/old_api
```

Check: `ls src tests docs` shows only `services`, `unit integration`, `api guides`; `find . -name 'old_*'` returns nothing.

Q30. Important files

0/20

You are working on a project where you need to back up important files to a separate backup directory. The project directory has the following structure:

```
project/
├── src/
│   ├── main.py
│   ├── utils.py
│   └── config/
│       └── settings.py
├── data/
│   ├── input.csv
│   └── output.csv
├── docs/
│   ├── README.md
│   └── requirements.txt
└── scripts/
    ├── deploy.sh
    └── test.sh
```

Your task is to:

Copy the entire src/ directory (including all its contents) to a backup directory named backup/ (backup directory will be created outside project directory).

Copy the docs/README.md file and scripts/deploy.sh file to the backup/ directory.

Ensure that the directory structure in backup/ mirrors the structure in project/, so the backup/ directory should look like this after the operation:

```
backup/
├── src/
│   ├── main.py
│   ├── utils.py
│   └── config/
│       └── settings.py
├── README.md
└── deploy.sh
```

```
Write the cp commands you would use to accomplish this task.


Note:

Make sure you are outside the backup directory before running the code.
```

Answer

Run from inside `project/`; `backup/` sits one level up, so use `../backup`.

```
mkdir -p ../backup

cp -r src ../backup/

cp docs/README.md scripts/deploy.sh ../backup/
```

Check: `ls -R ../backup` -> `src/{main.py,utils.py,config/settings.py}`, `README.md`, `deploy.sh`.

Q31. The Internship Application Folder

0/20

You are preparing to apply for a summer internship. Your resume and application documents are scattered across the current directory, making them difficult to manage.

Organize your application documents into a dedicated folder while keeping the original resume in the current directory.

Tasks:

Perform the following steps in the current directory:

Create a directory named Internship.

Copy resume.pdf into Internship, keeping the original file unchanged in the current directory.

Move cover_letter.txt into Internship.

Rename hobbies.txt to personal_interests.txt in the current directory.

Create an empty file named checklist.txt inside Internship.

Starting Structure:

```
├── resume.pdf
├── cover_letter.txt
└── hobbies.txt
```

Expected Structure:

```
├── resume.pdf
├── personal_interests.txt
└── Internship/
    ├── resume.pdf
    ├── cover_letter.txt
    └── checklist.txt
```

Requirements:

```
The original resume.pdf must remain in the current directory, and its copy must exist inside
Internship.

cover_letter.txt must exist only inside Internship.

hobbies.txt must be renamed to personal_interests.txt, preserving its contents.

checklist.txt must be an empty file.

Do not delete or modify the contents of the original files.
```

Answer

Create the folder, copy resume (keep original), move cover letter, rename hobbies, add empty checklist.

```
mkdir Internship

cp resume.pdf Internship/

mv cover_letter.txt Internship/

mv hobbies.txt personal_interests.txt

touch Internship/checklist.txt
```

Check: `ls -l` shows `resume.pdf personal_interests.txt Internship`; `ls Internship` shows `resume.pdf cover_letter.txt checklist.txt` (size 0).

Q32. Giving Permissions

0/20

You are working on a shared project where multiple users have different roles and permissions. You need to set the correct permissions on various files and directories to ensure that they are accessed only as intended.

The project directory structure is as follows:

```
project/
├── src/
│   ├── main.py
│   ├── utils.py
│   └── config.py
├── data/
│   ├── input_data.csv
│   └── output_data.csv
├── docs/
│   └── README.md
└── scripts/
    ├── run.sh
    ├── deploy.sh
    └── test.sh
```

Your tasks are:

Ensure that all Python files in the src/ directory (.py files) are executable only by the file owner. To play it safe, also provide read and write permissions for the owner.

Make the data/ directory and its contents are readable, writable, and executable by the owner and the group, but not accessible by others.

Make README.md readable by everyone but writable only by the owner.

Make all the script files in the scripts/ directory (.sh files) readable and executable by everyone but writable only by the owner.

Write the chmod commands you would use to accomplish these tasks.

```
Note:

Make sure that you are outside the project directory before running the code.
```

Answer

From the parent of `project/`, apply the octal mode matching each rule (owner rwx = etc.).

```
chmod 700 project/src/*.py

chmod -R 770 project/data

chmod 644 project/docs/README.md

chmod 755 project/scripts/*.sh
```

Check: `stat -c '%a %n' project/src/*.py project/data project/data/* project/docs/README.md project/scripts/*.sh` -> 700, 770, 644, 755 respectively.

Q33. The Missing Project Credentials

0/40

You are setting up a web application for a college event. The application cannot connect to its database because the required database credentials are missing from the output file.

Your teammate tells you:

"The database password is stored in a hidden file somewhere inside the EventApp project. Find the file containing the database password and copy its contents into a new file in the project directory."

For example, if the hidden file contains:

DB_PASSWORD=EvT_x9Kp!7m

The output file should contain the same line:

DB_PASSWORD=EvT_x9Kp!7m

The EventApp directory already exists and contains multiple files and subdirectories. The credentials file is hidden and may be located inside a nested directory.

Note: The DB_PASSWORD value in the hidden credentials file is dynamically generated and may differ across test cases. Do not hardcode the password.

Tasks:

Perform the following steps:

Navigate into the EventApp directory and inspect its contents, including hidden files and subdirectories.

Locate and inspect the hidden credentials file containing the DB_PASSWORD entry.

Create a file named db_password.txt directly inside EventApp, containing the complete DB_PASSWORD entry (key and its value) exactly as it appears in the source file.

Requirements:

```
The password must be retrieved dynamically from the existing credentials file.

Do not hardcode the example password.

Do not modify or delete any existing files.
```

Answer

Enter the project, list hidden entries, grep all files for the key, and write the matching line to the output file (no hardcoding).

```
cd EventApp

ls -la

find . -name ".*" -type f

grep -rh '^DB_PASSWORD=' . > db_password.txt
```

Check: `cat db_password.txt` shows exactly one `DB_PASSWORD=...` line identical to the value in the hidden file.

Q34. Project Cleanup & Backup

0/60

You are organizing a project directory before submission. The Project directory contains source code, documentation, assets, temporary files, and a hidden configuration directory.

Your task is to create a backup of the required files, organize the documentation, and clean up the project while preserving the specified original files.

Initial Directory Structure:

```
Project/
├── src/
│   ├── app.py
│   └── old_app.py
├── docs/
│   ├── setup.txt
│   └── usage.txt
├── assets/
│   └── logo.png
├── draft.txt
├── temp.txt
└── .config/
    └── version.txt
```

Tasks:

Perform the following steps inside the Project directory:

Create an empty file named index.txt in the Project directory.

Create a directory named Backup and copy the entire src directory into it, preserving both files and their contents.

Move logo.png from assets into Backup.

Inspect the hidden files and directories inside Project. Locate the hidden configuration directory, find version.txt inside it, and copy the file into the root Project directory.

Rename draft.txt to README.txt and move it into Backup.

Combine the contents of docs/setup.txt and docs/usage.txt, in that order, into a file documentation.txt inside Backup folder. Preserve the contents of both files.

```
Append Backup completed as a new line at the end of Backup/documentation.txt.

Delete old_app.py from the original src directory and delete temp.txt.

Remove the assets and docs directories after handling their contents.

Create an empty file named backup.log inside Backup folder.


Expected Directory Structure:

Project/

├── index.txt

├── version.txt

├── src/

│   └── app.py

├── .config/

│   └── version.txt

└── Backup/

    ├── src/

    │   ├── app.py

    │   └── old_app.py

    ├── logo.png

    ├── README.txt

    ├── documentation.txt

    └── backup.log
```

Answer

Work inside `Project/`; build `Backup/`, pull in the hidden config version, merge docs, then purge originals.

```
cd Project

touch index.txt

mkdir Backup

cp -r src Backup/

mv assets/logo.png Backup/

ls -la

cp .config/version.txt .

mv draft.txt Backup/README.txt

cat docs/setup.txt docs/usage.txt > Backup/documentation.txt

echo 'Backup completed' >> Backup/documentation.txt

rm -f src/old_app.py temp.txt

rm -rf assets docs

touch Backup/backup.log
```

Check: `find . | sort` matches the expected tree; `cat Backup/documentation.txt` ends with `Backup completed`.

Q35. The Daily Study Tracker

```
0/20


You are maintaining a daily study tracker for your Systems & AI Essentials course. Your current
directory contains two files: study.txt, which stores your completed study sessions, and temp.txt,
which contains your latest study entries.


Combine the new entries with your existing tracker, remove the temporary file, and organize the
tracker into a separate directory.


Tasks:


Perform the following steps in the current directory:


Append the contents of temp.txt into study.txt, preserving the existing entries.

Delete temp.txt after appending its contents.

Create a directory named StudyNotes.

Move study.txt into the StudyNotes directory.


Initial Directory Structure:

├── study.txt

└── temp.txt

Expected Directory Structure:

└── StudyNotes/

    └── study.txt
```

Answer

Append temp entries, delete temp, then move the tracker into StudyNotes.

```
cat temp.txt >> study.txt

rm temp.txt

mkdir StudyNotes

mv study.txt StudyNotes/
```

Check: `ls` shows only `StudyNotes`; `ls StudyNotes` shows `study.txt` containing the old plus new entries.

Q36. Create a Private Diary

```
10/20



You want a personal space on a shared machine where nobody else can read your notes. You will
create the file yourself and lock it down using chmod.



Scenario:



There is no existing structure. You must build everything from scratch in your current directory.



Tasks to Complete:



Create a directory called personal_space.

Inside it, create a file called diary.txt containing exactly the text Top Secret Entry.

Using chmod, set the permissions of diary.txt so that only the owner can read and write it. Group
and others must have no permissions at all.
```

Answer

Create the folder and file from scratch, then restrict it to owner read+write only.

```
mkdir personal_space

printf 'Top Secret Entry\n' > personal_space/diary.txt

chmod 600 personal_space/diary.txt
```

Check: `ls -l personal_space` shows `-rw-------`; `cat personal_space/diary.txt` prints `Top Secret Entry`.

Q37. Archiving a Release Directory

```
40/40


You are given the following directory structure:


release/

├── app.js

├── config.json

└── README.md


Requirements

Create a directory named archive.

Copy the entire release directory into archive.

Rename the copied directory from release to release_v1.

Remove the original release directory.
```

Answer

Copy the tree into `archive/`, rename the copy, then delete the original.

```
mkdir archive

cp -r release archive/

mv archive/release archive/release_v1

rm -rf release
```

Check: `ls archive` shows only `release_v1`; `ls release_v1` shows the three files; `ls -d release` fails.

Q38. Organize and Review Study Notes

```
20/20


You want to organize your study notes for different subjects and quickly review them from the
terminal.


notes/

├── linux/

│ └── notes.txt

├── git/

│ └── notes.txt

└── javascript/

    └── notes.txt


Task

Add a heading to each notes.txt file:

Linux Notes in linux/notes.txt

Git Notes in git/notes.txt

JavaScript Notes in javascript/notes.txt

Append the content of all notes files in notes/final.txt.
```

Answer

Insert a subject heading as the first line of each notes file (GNU `sed -i`), then concatenate all three into `final.txt`.

```
sed -i '1i Linux Notes' notes/linux/notes.txt

sed -i '1i Git Notes' notes/git/notes.txt

sed -i '1i JavaScript Notes' notes/javascript/notes.txt

cat notes/linux/notes.txt notes/git/notes.txt notes/javascript/notes.txt > notes/final.txt
```

Check: `head -1 notes/*/notes.txt` shows each heading; `wc -l notes/final.txt` equals the sum of the three files.

Q39. Project Treasure Hunt

```
0/20

You receive a mysterious envelope containing an old map and a cryptic note hinting at buried
treasures. Following the map leads you to a computer terminal, where you must navigate a directory
to uncover the next clue.



Folder structure:



home/user/treasure_hunt

├── clues

│   ├── first_clue.txt

│   └── second_clue.txt

├── history

│   ├── year_2000

│   │   └── event.txt

│   └── year_1990

└── notes

    └── secrets.txt


Task Instructions:

Navigate to the treasure_hunt directory located inside the user directory in the home directory.

Inside the history folder, create a new directory named year_1980.

Inside the clues folder, create a new empty text file named third_clue.txt.

Inside the notes folder, create two new empty text files: reminder.txt and next_step.txt.

Use relevant terminal-based commands to complete the tasks.




Note:


Get rid of all the debugging code at last to have the test cases passed.
```

Answer

Change into the treasure_hunt directory, then add the new directory and empty files.

```
cd ~/user/treasure_hunt

mkdir history/year_1980

touch clues/third_clue.txt

touch notes/reminder.txt notes/next_step.txt
```

Check: `ls history clues notes` shows `year_1980`, `third_clue.txt`, and `reminder.txt next_step.txt secrets.txt`.

Q40. ProjectNST

```
0/20



You have just joined a new software development team, and your first task is to add a new project called Capstone_Project to the workspace ProjectNST.



The existing folder structure is:



│

├── Dev_Project

│

├── Portfolio

│

└── WindowSwap

    └── README.txt




The lead developer gives you the following responsibilities:



Add the Project: Start by creating a directory named Capstone_Project for the project inside the ProjectNST workspace so all files can be neatly stored.

Provide Documentation: The team requires every project folder to include a README.txt file with a brief description.

Include README.txt file inside each project -> Portfolio, Dev_Project, and Capstone_Project. The text inside README.txt must be according to the project, in the given format, ProjectName Documentation. Eg: for Project Portfolio, README.txt file must contain Portfolio Documentation.
```

Answer

Run inside `ProjectNST/` : create the Capstone folder and write one README per project using the required format.

```
mkdir Capstone_Project

printf 'Portfolio Documentation\n' > Portfolio/README.txt

printf 'Dev_Project Documentation\n' > Dev_Project/README.txt

printf 'Capstone_Project Documentation\n' > Capstone_Project/README.txt
```

Check: `cat Portfolio/README.txt Dev_Project/README.txt Capstone_Project/README.txt` each print exactly `<ProjectName> Documentation` .

Q41. Library Organization

0/20

In a futuristic society, the Digital Library of Alexandria holds vast collections of ancient and modern knowledge. However, due to some recent changes, the library's digital archive has become somewhat disorganized. As a digital librarian, your task is to reorganize some key files and collections, move entire sections and ensure everything is in the right place.

Current Folder Structure:

```
DigitalLibrary
├── Ancient
│   ├── scrolls.txt
│   ├── manuscripts.txt
├── Modern
│   ├── novels.txt
│   ├── essays.txt
│   ├── plays.txt
├── TemporaryFiles
│   ├── backup_novels.txt
│   ├── backup_plays.txt
```

Note: You are currently right outside the DigitalLibrary folder.

Tasks:

Copy Files:

Create a new folder called AncientTexts right inside the DigitalLibrary folder and copy the file scrolls.txt from the Ancient folder into the AncientTexts folder.

Move a Folder:

Create a new folder called Archives right inside the DigitalLibrary folder and move the entire Modern folder inside the Archives folder so that all modern literature is stored under the Archives folder.

Organize Backup Files:

Move the files backup_novels.txt and backup_plays.txt from TemporaryFiles inside the Modern folder that you just moved inside the Archives folder.

```
Final Folder Structure:


DigitalLibrary
├── Ancient
│   ├── scrolls.txt
│   ├── manuscripts.txt
├── AncientTexts
│   ├── scrolls.txt
├── Archives
│   ├── Modern
│   │   ├── novels.txt
│   │   ├── essays.txt
│   │   ├── plays.txt
│   │   ├── plays.txt
│   │   ├── backup_novels.txt
│   │   ├── backup_plays.txt
├── TemporaryFiles
```

Answer

Work from outside `DigitalLibrary/`; create the two folders, copy below `scrolls.txt`, move `Modern` under `Archives`, then move the backups into it.

```
mkdir DigitalLibrary/AncientTexts

cp DigitalLibrary/Ancient/scrolls.txt DigitalLibrary/AncientTexts/

mkdir DigitalLibrary/Archives

mv DigitalLibrary/Modern DigitalLibrary/Archives/

mv DigitalLibrary/TemporaryFiles/backup_novels.txt DigitalLibrary/TemporaryFiles/backup_plays.txt
DigitalLibrary/Archives/Modern/
```

Check: `find DigitalLibrary | sort` shows `AncientTexts/scrolls.txt`, `Archives/Modern/{novels,essays,plays,backup_novels,backup_plays}.txt`, and an emptied `TemporaryFiles/`.

Practice set B (12 questions)

Mixed drills — second pass. — 12 questions.

Q42. Recovering a Lost API Key

0/20

You have joined a team that is testing a weather application. The application keeps failing with the following error:

Error: API_KEY not found

A senior developer tells you:

"The API key is already stored somewhere in the project directory. We usually keep sensitive configuration in hidden files. Find the API key and save it to a file named api_key.txt."

Task:

Explore the projects directory.

Find the hidden file that contains the API key.

Create a file named api_key.txt in the projects directory containing only the API key value.

Example Project Structure:

```
projects/
├── app.js
├── README.md
├── logs/
├── src/
└── .env
```

Requirements:

Do not modify the hidden file containing the API key.

The file api_key.txt must contain only the API key value.

No extra text, spaces, or additional lines should be added.

Expected File Content (api_key.txt):

<API_KEY_VALUE>

Answer

The key lives in the hidden `.env` as `API_KEY=...`; strip the variable name and write only the value (no trailing newline).

```
cd projects

grep -n 'API_KEY' .env

printf '%s' "$(grep -E '^API_KEY=' .env | cut -d= -f2-)" > api_key.txt
```

Check: `cat api_key.txt` prints only the key (no label, no newline); `.env` is unchanged.

Q43. Inspect Repository Status and Commit a Specific File

```
0/20



Repo has two files: main.py (modified) and README.md (unmodified). Run git status first to inspect.
Then stage only main.py and commit with message: Update main script.
```

Answer

Inspect first, then stage only `main.py` and commit it.

```
git status

git add main.py

git commit -m "Update main script"
```

Check: `git status` shows a clean tree and `git log --oneline -1` shows "Update main script".

Q44. Cleanup and Organize Documents

```
0/60


You have a directory Office with the following structure:


Office/
├── Reports/
│   ├── jan.txt
│   ├── feb.txt
│   └── mar.txt
├── old_notes.txt
└── draft.txt


Perform the following tasks :


Create a new empty file named Office/index.txt.

Delete the file old_notes.txt from Office — it is outdated.

Rename draft.txt to final.txt inside Office.

Delete the file Office/Reports/mar.txt — it's no longer required.

Create a new folder named Office/Reports_Backup, and inside it, combine the contents of
Office/Reports/jan.txt and Office/Reports/feb.txt into a new file named q1_summary.txt.

Create an empty file named Office/Reports_Backup/backup_created.txt to mark the backup.
```

Answer

Run from the parent of `Office/`: create/delete/rename first, then build the backup directory and concatenate in order.

```
touch Office/index.txt

rm Office/old_notes.txt

mv Office/draft.txt Office/final.txt

rm Office/Reports/mar.txt

mkdir -p Office/Reports_Backup

cat Office/Reports/jan.txt Office/Reports/feb.txt > Office/Reports_Backup/q1_summary.txt

touch Office/Reports_Backup/backup_created.txt
```

Check: `ls -R Office` shows no `old_notes.txt/mar.txt`, `final.txt` exists, and `q1_summary.txt` starts with jan then feb.

Q45. Updating an Existing Application Log

```
0/20


You are helping your team debug a web application. The application already maintains a log file
named app.log that contains important startup information.


Before you begin, the file already contains:


Server Initializing...

Server Started




A user has just successfully logged in, and you need to record this event in the log file.


Task:


Add the following line to the end of app.log:


User login successful


Requirements:

Do not delete or modify the existing contents of app.log.

Append the new log entry to the file.


Expected File Content:

Server Initializing...

Server Started

User login successful
```

Answer

Append one line without touching existing content.

```
echo "User login successful" >> app.log
```

Check: `tail -3 app.log` shows the two original lines followed by "User login successful".

Q46. Banking Records Under Lock

0/60

You are an internal auditor at a national bank. The first-quarter ledgers must be reviewed, archived, consolidated into a secure vault, and temporary files cleaned up. The bank's file system is locked down: some directories cannot be written to, and the vault cannot even be entered. You must work out exactly which permissions each task needs. Some tasks need no permission changes at all, and some need a temporary change to a directory. Every temporary change must be reverted, and every final permission must match the requirements exactly.

Given Directory Structure:

```
bank/
├── ledgers/
│   ├── jan.txt
│   ├── feb.txt
│   └── mar.txt
├── vault/
├── reports/
├── tmp/
│   ├── a.tmp
│   ├── b.tmp
│   ├── c.tmp
│   └── session.log
└── master.key
```

Pre-existing Permissions:

ledgers/: read and execute for owner, group and others; no one has write permission.

jan.txt, feb.txt, mar.txt: read-only for owner, group and others.

vault/: no permissions for anyone.

reports/: read, write and execute for owner; read and execute for group and others.

tmp/: read and execute for owner, group and others; no one has write permission.

a.tmp, b.tmp, c.tmp, session.log: read and write for owner; read-only for group and others.

master.key: no permissions for anyone.

Actions to Perform:

Copy February's ledger for review.

Copy ledgers/feb.txt into reports/ as feb_copy.txt.

A copied file does not automatically carry the original's permissions. Set feb_copy.txt to read and write for owner, read-only for group, and no permissions for others.

Annotate February's ledger.

Append the exact line FEB-28 Reviewed by auditor to ledgers/feb.txt.

After appending, feb.txt must be read-only for owner, group and others again.

Update the session log.

Append the exact line Audit session opened to tmp/session.log.

Create the archive.

Inside reports/, create the nested path archive/2026/.

The 2026/ directory must have read, write and execute for owner, read and execute for group, and no permissions for others.

Archive the review copy.

Move reports/feb_copy.txt into reports/archive/2026/ and rename it to feb_review.txt in the same command.

Tag and secure the reports.

Create an empty hidden file reports/.audit_tag with no permissions for owner, group or others.

Set reports/ to read, write and execute for owner, read and execute for group, and no permissions for others.

Consolidate the ledgers in the vault.

Create vault/q1_ledger.txt containing the full contents of jan.txt, feb.txt and mar.txt, in exactly that order. The q1_ledger must be readable only by the owner, with no write permission for anyone.

At the end, vault/ must have no permissions for anyone, exactly as it was at the start.

Purge temporary files.

Delete all three .tmp files inside tmp/ using a single wildcard command. session.log must not be deleted.

At the end, tmp/ must have exactly its original permissions.

Important:

master.key must not be touched. At the end it must still have its original content and no permissions for anyone.

feb_review.txt must not contain the line added in step 2, but q1_ledger.txt must contain it.

```
session.log must still exist with its original permissions, containing its original content plus
the line from step 3.

jan.txt and mar.txt must remain unchanged, with their original content and permissions.

Order matters. Read all the actions before you begin.
```

Answer

Copy the ledger before annotating it, temporarily unlock only what each step needs, and restore every directory/file permission at the end.

```
cd bank

cp ledgers/feb.txt reports/feb_copy.txt

chmod 640 reports/feb_copy.txt



chmod u+w ledgers/feb.txt

echo "FEB-28 Reviewed by auditor" >> ledgers/feb.txt

chmod 444 ledgers/feb.txt



echo "Audit session opened" >> tmp/session.log



mkdir -p reports/archive/2026

chmod 750 reports/archive/2026

mv reports/feb_copy.txt reports/archive/2026/feb_review.txt



touch reports/.audit_tag

chmod 000 reports/.audit_tag

chmod 750 reports



chmod 700 vault

cat ledgers/jan.txt ledgers/feb.txt ledgers/mar.txt > vault/q1_ledger.txt

chmod 400 vault/q1_ledger.txt

chmod 000 vault



chmod u+w tmp

rm tmp/*.tmp

chmod 555 tmp
```

Check: `grep FEB-28 reports/archive/2026/feb_review.txt` fails, but `grep FEB-28 vault/q1_ledger.txt` succeeds; `ls -ld vault tmp ledgers reports` and `stat -c '%a %n' master.key tmp/session.log` match the originals (000, 644).

Q47. Multiple File Commits and Commit History Review

```
0/20


Repository is already initialized.


The files header.html and footer.html already exist in the repository.


Add some content to header.html, then stage and commit your changes with the message:


Add header


Next, add some content to footer.html, then stage and commit your changes with the message:


Add footer


Finally, run:


git log --oneline


to verify that both commits appear in the commit history.
```

Answer

Edit each file, stage and commit it separately, then review the log.

```
echo "<header content>" >> header.html

git add header.html

git commit -m "Add header"


echo "<footer content>" >> footer.html

git add footer.html

git commit -m "Add footer"


git log --oneline
```

Check: `git log --oneline` lists "Add footer" above "Add header" (newest first).

Q48. Selective Staging with Multiple Commits and Log Review

```
0/40


A Git repository is already initialized, and both README.md and main.py already exist.


Add some content to README.md, stage only README.md, and commit it with the message:


Update readme


Then add some content to main.py, stage only main.py, and commit it with the message:


Update main


Finally, run:


git log --oneline


to view the commit history.
```

Answer

Stage one file per commit; never use `git add .`.

```
echo "<readme content>" >> README.md

git add README.md

git commit -m "Update readme"


echo "<main content>" >> main.py

git add main.py

git commit -m "Update main"


git log --oneline
```

Check: `git log --oneline` shows "Update main" then "Update readme"; `git status` is clean.

Q49. Three Levels of Access

```
10/20


After a security review, three files in the clearance directory were found with wide-open
permissions. Each file must be tightened to a different clearance level using numeric chmod.


Given Directory Structure:


clearance/
├── public.txt
├── team.txt
└── private.txt


Current Permissions:


All three files: full permissions for everyone.


Tasks to Complete:


public.txt: owner can read and write; group and others can only read.
team.txt: owner can read and write; group can only read; others get nothing.
private.txt: owner can read and write; group and others get nothing.
Do not modify the content of any file.
```

Answer

Use numeric modes: 644 (rw-r--r--), 640 (rw-r-----), 600 (rw-------).

```
chmod 644 clearance/public.txt
chmod 640 clearance/team.txt
chmod 600 clearance/private.txt
```

Check: `ls -l clearance` shows `-rw-r--r--`, `-rw-r-----`, `-rw-------` respectively.

Q50. Purge the Locked Workspace

```
0/40



A corrupted draft is stuck inside a locked workspace. The draft itself is read-only, and the
workspace directory has no write permission. Deleting a file requires write permission on the
directory, not on the file. Delete the corrupted draft, create a clean replacement, and lock the
workspace again.



Given Directory Structure:



workspace/     (Write access removed)

├── old_draft.txt   (Read-only, corrupted)

└── notes.txt



Current Permissions:



workspace/ directory: Write permission removed (chmod 555).

old_draft.txt: Read-only for everyone (chmod 444).



Tasks to Complete:



Delete workspace/old_draft.txt.

Create a new file workspace/final_draft.txt containing exactly Clean copy.

Using chmod, set the permissions of final_draft.txt to read and write for user, read for group and
no permissions for others.

notes.txt must remain untouched.

Restore workspace/ permissions to read and execute for user, group and others.
```

Answer

Temporarily add write on the directory to unlink the draft, then create the replacement and lock it back to 555.

```
chmod u+w workspace

rm workspace/old_draft.txt

printf '%s' 'Clean copy' > workspace/final_draft.txt

chmod 640 workspace/final_draft.txt

chmod 555 workspace
```

Check: `ls -l workspace` shows `final_draft.txt` only (plus `notes.txt`), mode `-rw-r-----`, and `workspace` is `dr-xr-xr-x`; `cat workspace/final_draft.txt` prints `Clean copy`.

Q51. Deployment Project Permissions

20/40

You are preparing a shared deployment project where different roles (developers, CI system, general viewers) need different levels of access to different parts of the codebase. You need to set the correct permissions on each part of the project so it is accessed only as intended.

Given Directory Structure:

```
deployment/
├── app/
│   ├── server.py
│   ├── handler.py
│   └── settings.py
├── datasets/
│   ├── train.csv
│   └── test.csv
├── docs/
│   └── OVERVIEW.md
└── bin/
    ├── start.sh
    ├── stop.sh
    └── healthcheck.sh
```

All files currently have default permissions (644 for files, 755 for directories). Apply the following rules:

Python source files (every .py file inside deployment/app/):

Owner: read, write, execute

Group: no access

Others: no access

The datasets/ directory, and every file inside it:

Owner: read, write, execute

Group: read, write, execute

Others: no access

```
docs/OVERVIEW.md:

Owner: read, write

Group: read only

Others: read only

Every shell script (every .sh file inside deployment/bin/):

Owner: read, write, execute

Group: read, execute (no write)

Others: read, execute (no write)


Deliverable: Write the chmod command(s) you would run, one rule at a time, to reach the end state
described above. You may use wildcards (e.g. *.py) instead of listing files individually.


Note: Run your commands from outside the deployment/ directory (i.e. paths should start with
deployment/..., not be run from inside it).
```

Answer

Apply one numeric mode per rule using wildcards; run from outside `deployment/`.

```
chmod 700 deployment/app/*.py

chmod 770 deployment/datasets deployment/datasets/*

chmod 644 deployment/docs/OVERVIEW.md

chmod 755 deployment/bin/*.sh
```

Check: `ls -l deployment/app deployment/datasets deployment/docs deployment/bin` shows `-rwx------`, `drwxrwx---`/`-rwxrwx---`, `-rw-r--r--`, and `-rwxr-xr-x` respectively.

Q52. Audit Trail Setup

```
10/20


You are given the following folder structure inside the current directory.


audit_trail/

└── access.log


Perform the following operations:


Change the permissions of the audit_trail/ directory so that the owner has full access (read,
write, execute), the group has read and execute only, and others have no access at all.

Inside audit_trail/, create a new file named summary.txt containing the exact text Audit Summary
Generated.

Set the permissions of summary.txt so that the owner can read and write, the group can only read,
and others have no access.
```

Answer

Set the directory to 750, create the summary, then set it to 640.

```
chmod 750 audit_trail

printf '%s' 'Audit Summary Generated' > audit_trail/summary.txt

chmod 640 audit_trail/summary.txt
```

Check: `ls -ld audit_trail` shows `drwxr-x---` and `ls -l audit_trail/summary.txt` shows `-rw-r-----`; `cat` prints only the required text.

Q53. Data Breach Response

0/60

You have been tasked with handling Data Breach Response, a secure directory of evidence and reports from an active security incident. Some files and directories currently have restricted permissions that you must work around without permanently weakening security.

Given Directory Structure:

```
breach_response/
├── evidence/
│   └── log_dump.txt
├── secure/
├── reports/
│   └── incident_summary.txt
└── classified.txt
```

Pre-existing Permissions:

evidence/ directory — write permission removed (chmod 555)

log_dump.txt — read-only (chmod 444)

incident_summary.txt — read-only (chmod 444)

classified.txt — no permissions (chmod 000)

Actions to Perform:

Copy a file into breach_response/:

Copy evidence/log_dump.txt to breach_response/log_dump.txt.

Move the restricted directory:

Move the entire evidence/ directory into secure/, resulting in secure/evidence/.

evidence/ currently has no write permission (555). Since moving a directory into a new parent requires updating its own metadata, you'll need to temporarily add write permission to evidence/ before moving it.

After the move, restore secure/evidence/ to permission 555.

```
Edit the read-only report:

Append the exact line Breach confirmed to reports/incident_summary.txt.

incident_summary.txt currently has no write permission (444), so you'll need to temporarily adjust
permissions to edit it.

Restore the file permissions to 444 afterward.

Create a new report:

Create secure/forensics_report.txt containing the exact text Forensics Complete.

Set its permissions to 600.

Append to the report again:

Append the exact line Escalated to security team to reports/incident_summary.txt.

As before, temporarily adjust permissions to write, then restore incident_summary.txt to 444
afterward.



Important: Do not modify classified.txt. It must remain unchanged with permission 000.
```

Answer

Copy first, unlock each protected item only for the operation that needs it, then restore 555/444 immediately; leave `classified.txt` alone.

```
cp breach_response/evidence/log_dump.txt breach_response/log_dump.txt


chmod u+w breach_response/evidence

mv breach_response/evidence breach_response/secure/

chmod 555 breach_response/secure/evidence


chmod u+w breach_response/reports/incident_summary.txt

echo "Breach confirmed" >> breach_response/reports/incident_summary.txt

chmod 444 breach_response/reports/incident_summary.txt


printf '%s' 'Forensics Complete' > breach_response/secure/forensics_report.txt

chmod 600 breach_response/secure/forensics_report.txt


chmod u+w breach_response/reports/incident_summary.txt

echo "Escalated to security team" >> breach_response/reports/incident_summary.txt

chmod 444 breach_response/reports/incident_summary.txt
```

Check: `ls -ld breach_response/secure/evidence` is `dr-xr-xr-x`, `incident_summary.txt` is `-r--r--r--` with both lines, `forensics_report.txt` is `-rw-------`, and `classified.txt` is still `----------` with unchanged content.

PART 3 — IN-CLASS QUIZ MCQs (23, with answers)

Answer key (23 MCQs)

Module	Question	Answer	Why
M1	Which statement best describes a computer?	**A machine that follows changeable instructions**	A computer executes instructions that can be changed/updated — not just a screen+keyboard or internet device.
M1	In a smartphone, which action is an example of input?	**Tapping an icon on the screen**	Touch sends information into the device; display/storage/vibration are output or internal.
M1	After a computer processes data, what happens next?	**Output is produced for the user**	IPOS: input → processing → output → storage.

Module	Question	Answer	Why
M1	Why is a juicer not considered a computer?	**It cannot be programmed to perform different tasks**	Fixed mechanical action is not programmability.
M1	Which device is best classified as an embedded computer?	**Smartwatch tracking daily steps**	Dedicated single-purpose computing system inside a device.
M1	Which example best represents storage in an MRI system?	**Scan reports saved in records**	Saving for later = storage; display/process/waves are not.
M1	Why do data centres use thousands of computers together?	**To support very large workloads**	Scale for massive compute/storage demand.
M1	Which feature makes a smartwatch different from a laptop?	**Smartwatch is designed mainly for limited and specific tasks**	Embedded/special-purpose vs general-purpose.
M1	A self-checkout machine prints a receipt after scanning items. The receipt is:	**Output produced by the system**	Information produced for the user.
M1	What mainly separates a phone from a data centre?	**Scale of operation and primary purpose**	Both compute/store/connect; scale and purpose differ.
M1	Temporary working memory inside a computer is called _____ (3 letters).	**RAM**	Volatile working memory while programs run.
M1	Long-term file storage in modern laptops is often an _____ (3 letters).	**SSD**	Non-volatile storage; keeps files when powered off.
M1	Every computer must follow input, processing, output, and storage. (T/F)	**true**	The IPOS model applies to all computers.
M1	A mechanical juicer is programmable like a computer. (T/F)	**false**	No instruction execution — fixed action only.
M2	Which component executes instructions in a computer?	**CPU**	Fetch-decode-execute cycle.
M2	Which component is mainly responsible for graphics rendering and parallel processing?	**GPU**	Many cores tuned for parallel work.
M2	Which hardware component connects all major components together?	**Motherboard**	Main circuit board / bus routing.
M2	Which of the following is an input device?	**Keyboard**	Enters data; speaker/monitor/projector are output.
M2	You open 20 browser tabs and your laptop becomes slow. Which component are you running out of?	**RAM**	Each tab holds state in RAM.
M2	Which of the following is an output device?	**Monitor**	Displays information to the user.

Module	Question	Answer	Why	
M3	Which command makes colourful rainbow output in the terminal?	**lolcat**	`echo hello	lolcat `(alias of cat` + ANSI colours).
M3	In cowsay, what does the -f flag do? (e.g. cowsay -f dragon "Deploy successful")	**Modifies the command behaviour (chooses the dragon template)**	It is a flag/option, not a file or template creator.	
M6	A CPU can directly execute which of the following?	**Machine code instructions represented in binary**	Source code must be compiled/interpreted to machine code first.	

> Raw captures of these quiz pages (with portal option ordering) are in `psp_qa`-style dumps under `materials/lecture-questions/eq7lnkj6lo7s_*.md`.

Revision quizzes

The 11 graded revision quizzes (schedules only)

Questions are hidden until **Start Test** — never started, so no text exists. Links live in the portal Quiz tab for subject `eq7lnkj6lo7s`.

Quiz	Window	XP
Course & Instructor Intro, Computer Systems	1–6 Sep	32/38
Computer Hardware, Software, OS	3–8 Sep	0/26
Command-Line Interface (CLI), Terminal	8–13 Sep	16/16 ✅
CLI File/Dir, Package Mgmt, Flags	10–15 Sep	28/26 ✅
OS, OS Architecture, Device Mgmt, Process Mgmt	16–21 Sep	0/34
OS, Filesystem, Ownership, PATH, os module	22–27 Sep	0/24
Software, Compiler, Interpreter, Number Rep, Encoding	24–29 Sep	0/48
Number Systems, Decimal, Binary, Hex, Conversion	29 Sep–4 Oct	13/30
Version Control, Git, Git Commands	1–6 Oct	24/24 ✅
GitHub, Remotes, Collaboration, PRs	6–11 Oct	0/34
Computer Networks, LAN/MAN/WAN, DNS, Packets, Protocols, Routers	19–24 Oct	0/22

Highest-value gaps: Computer Networks (0/22, newest), GitHub (0/34), Software+Number systems (0/48 — biggest XP), OS architecture (0/34).

PART 4 — GAPS & SUFFICIENCY

Area	Status
All 53 portal assignment questions	✅ full text + model answers
23 in-class quiz MCQs	✅ full text + answers (portal's own explanations where present)
11 revision quizzes (~250 XP of questions)	❌ text hidden behind Start Test — links listed above
Quiz contests	not present for this subject beyond Contest 1 (coding, solved in Q&A)
Module 1–2 (computers, hardware)	⚠️ no portal questions at all — concept sheet is your only source; read M1–M2 carefully
Networks (M10)	⚠️ 1 quiz only, no assignment questions — concept sheet + M10 one-liners are the whole prep

Sufficiency verdict: for the *question pool* this kit is complete (53/53 + 23/23). For *coverage of the syllabus* the weak spots are **Module 1–2 and Module 10** (networks) — no portal practice exists, so learn those from Part 1.